

n8n ON macOS — NO-CODE AI AGENTS

Name: _____ Branch/Year: _____ Date: _____

Warm-Up: What Do You Already Know?

1. What does "No-Code" mean in software development?

- A. Writing code in a language the computer doesn't understand
- B. Building applications using visual interfaces instead of writing code
- C. Deleting all the code from a project
- D. Using only HTML and CSS without JavaScript

Hint: *Think about what "no code" literally means — no writing of code!*

2. What is an "AI Agent"?

- A. A robot that walks around an office
- B. An AI system that can perceive its environment, make decisions, and take actions autonomously
- C. A human who specializes in artificial intelligence
- D. A chatbot that can only answer pre-programmed questions

Hint: *The key word is "agent" — something that acts on its own.*

3. What is an API?

- A. A programming language used for AI
- B. A way for two software systems to talk to each other
- C. A type of database
- D. An AI model made by Google

Hint: *API stands for Application Programming Interface — it's a bridge between systems.*

4. What is Node.js?

- A. A front-end CSS framework
- B. A JavaScript runtime that lets you run JS outside the browser
- C. A Python library for machine learning
- D. A cloud hosting service

Hint: *The name "Node.js" has "JS" in it — what language is that?*

Part 1: The No-Code Revolution — Why n8n?

THE SCENARIO: "The Overworked Developer"

Imagine you're a developer at a startup. Every morning, your boss wants a daily email with a summary of trending news, filtered by topic, and explained by AI. The traditional approach? Write a Python script with `requests`, `smtpplib`, API keys, cron jobs, error handling, logging... That's **200+ lines of code** and hours of debugging.

Now imagine doing the **exact same thing** by dragging 5 boxes on a screen and connecting them with arrows. No code. No debugging. Done in 15 minutes.

That's n8n. That's the No-Code revolution.

What is n8n?

n8n (pronounced "nodemation") is a **free, open-source, low-code/no-code workflow automation tool**. It lets you build AI agents, automate tasks, and connect different apps — all by visually connecting "nodes" on a canvas.

THE ANALOGY: LEGO Blocks for Software

Think of n8n like **LEGO blocks**. Each block (node) does one specific thing — fetch data, filter it, send an email, ask AI a question. You snap them together to build anything you want. No welding. No manufacturing. Just connect and go.

Why n8n Over Other Tools?

Feature	n8n	Zapier	Make (Integromat)
Pricing	Free & Open Source	Paid (free tier very limited)	Freemium (limited)
Self-Hosted?	Yes — runs on your laptop or server	No — cloud only	No — cloud only
AI/LLM Nodes	Built-in (Groq, OpenAI, etc.)	Limited AI support	Some AI modules
Custom Code	JavaScript & Python nodes	Very limited	Limited
Community	Large open-source community	Corporate support	Growing community
Data Privacy	Your data stays on YOUR machine	Data goes through Zapier servers	Data goes through Make servers

ACTIVITY: Think & Write:

Name 2 tasks in your daily life (or a future job) that you'd love to automate:

1. _____
2. _____

For each, would you prefer writing Python code or using a visual tool like n8n? Why?

KEY INSIGHT: n8n is **not a replacement** for coding — it's a **superpower on top of it**. Developers who know both coding AND no-code tools are 10x more productive. n8n handles the "plumbing" (APIs, scheduling, email) so you can focus on the logic.

Part 2: No-Code vs Code — What's the Difference?

THE ANALOGY: Cooking From Scratch vs. Using a Rice Cooker

Coding = Cooking rice on a stove. You control the flame, water ratio, timing — full control but more effort and room for error.

No-Code = Using a rice cooker. You put in rice and water, press a button — it handles the rest. Less control, but faster and more reliable for standard tasks.

Aspect	Traditional Code (Python/JS)	No-Code (n8n)
How you build	Write text-based instructions line by line	Drag, drop, and connect visual nodes
Skill needed	Programming language syntax, debugging	Logical thinking, understanding workflows
Speed to build	Hours to days	Minutes to hours
Flexibility	Unlimited — you control everything	Limited to available nodes & integrations
Debugging	Read error logs, trace stack traces	Click a node, see INPUT/OUTPUT visually
Maintenance	Update dependencies, fix breaking changes	Usually just re-connect or update a field
Best for	Complex logic, custom algorithms, ML models	Automations, integrations, AI agents, workflows

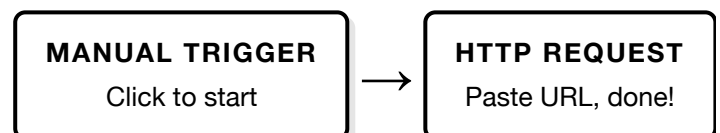
Side-by-Side Example: Fetch a Joke from an API

Python (5 lines of code)

```
import requests

response = requests.get(
    "https://official-joke-
api.appspot.com/random_joke"
)
joke = response.json()
print(f"Setup: {joke['setup']}")
print(f"Punchline: {joke['punchline']}")
```

n8n (2 nodes, zero code)



Same result. Zero syntax errors. Zero imports.

ACTIVITY: Fill in the Blanks:

No-code tools let you build by _____ and _____ visual nodes.

Traditional coding gives you more _____ but takes more _____.

n8n is best described as a _____ tool because it allows both visual building AND custom code nodes.

Hint: *low-code / no-code*

Quick Check: When should you use n8n instead of writing Python?

- A. When building a complex neural network from scratch
- B. When automating a workflow that connects APIs, AI, and email
- C. When writing a sorting algorithm for a data structures exam
- D. When building a custom game engine

Hint: *n8n excels at connecting different services together — think "plumbing" not "algorithms."*

Part 3: The Building Blocks of n8n

Before you build your first workflow, you need to understand the **6 core building blocks** of n8n. Think of these as your vocabulary — once you know them, you can "speak n8n" fluently.

THE ANALOGY: n8n is a Factory Assembly Line

Nodes = Machines on the line (each does one job)

Workflow = The full assembly line (machines connected in order)

Trigger = The ON button that starts the line

Connections = Conveyor belts between machines

INPUT/OUTPUT = Raw materials going in, finished product coming out

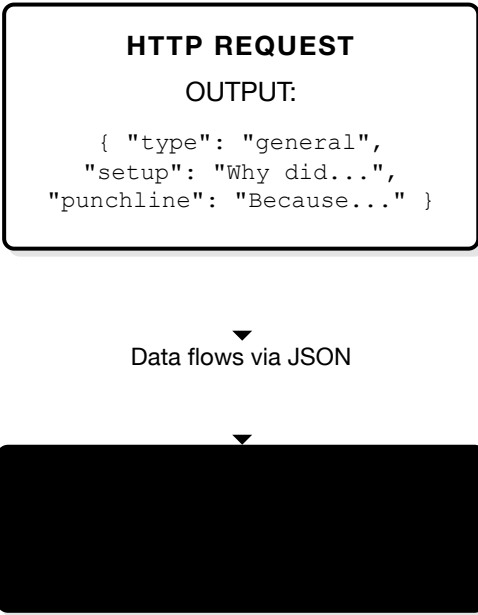
Credentials = Security badges to access restricted machines

Building Block	What It Does	Python/Code Equivalent	Example in n8n
Node	One unit of work — does a single task	A function or method	HTTP Request, Gmail, IF
Workflow	A connected sequence of nodes	A script/program (chain of functions)	Fetch Joke → Filter → Email
Trigger Node	Starts the workflow (the "ON" button)	<code>if __name__ == "__main__":</code>	Manual Trigger, Schedule Trigger
Connection (Arrow)	Passes data from one node to the next	Function return value → next function's input	Arrow between two nodes
INPUT / OUTPUT Panel	Data coming IN to a node / going OUT after execution	<code>print(input) / return output</code>	Click any node → see its data
Credentials	API keys/passwords stored securely	<code>os.getenv("API_KEY")</code>	Groq API key, Gmail OAuth

Data Flow in n8n — How Nodes Talk to Each Other

Every node receives data from the previous node as **JSON**. You reference fields from previous nodes using the syntax:

```
{{ $json.fieldName }}
```



ACTIVITY: Match the Building Blocks:

Draw a line connecting each n8n concept to its Python equivalent:

n8n Concept	Python Equivalent
1. Node	A. <code>if/else</code> statement
2. Workflow	B. <code>requests.get()</code>
3. IF Node	C. A function
4. HTTP Request Node	D. A complete Python script
5. Schedule Trigger	E. <code>cron job / schedule library</code>
6. <code>{{ \$json.field }}</code>	F. Accessing a dictionary key <code>data["field"]</code>

Your answers: 1→ _____ 2→ _____ 3→ _____ 4→ _____ 5→ _____ 6→ _____

Hint: A Node does one job (like a function). A Workflow is a full script. Think about what each Python concept does and match it to n8n.

KEY INSIGHT: Every n8n concept has a direct equivalent in Python/code. Learning n8n doesn't replace your coding skills — it **complements** them. You're learning the same logic, just in a visual language.

Part 4: Installation — Setting Up n8n on macOS

n8n requires **Node.js v18 or v20**. The default Homebrew Node (v25+) is too new and will cause errors. We use **nvm** (Node Version Manager) to install the correct version — similar to Python's virtual environments.

THE ANALOGY: nvm = Python's Virtual Environments

Just like you use `venv` or `conda` in Python to manage different Python versions, **nvm** lets you manage different Node.js versions. You can switch between Node 18, 20, or 22 with a single command.

Follow these 10 steps exactly:

Step 1 — Install nvm (Node Version Manager)

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

✔ You will see: `'Cloning into /Users/<you>/.nvm'` — this means it worked

Step 2 — Load nvm in your current terminal session

```
export NVM_DIR="$HOME/.nvm" && [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
```

💡 Alternatively, close the terminal and open a fresh one.

Step 3 — Make nvm load automatically every time (permanent)

```
cat >> ~/.zshrc << 'EOF'
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
EOF
```

Step 4 — Verify nvm is working

```
nvm --version
```

✅ Expected output: 0.39.7

Step 5 — Install Node.js v20 LTS

```
nvm install 20
```

✅ Expected: *Now using node v20.20.2 (npm v10.8.2)*

Step 6 — Verify Node version

```
node -v      # should show v20.x.x
npm -v       # should show v10.x.x
```

Step 7 — Uninstall any broken n8n (skip if first time)

```
npm uninstall -g n8n
```

Step 8 — Install n8n globally

```
npm install -g n8n
```

⚠️ Takes 2-4 minutes. Ignore `npm warn` messages.

Step 9 — Start n8n

```
n8n start
```

✅ You will see: *'Editor is now accessible via: http://localhost:5678'*

Step 10 — Open n8n in your browser

Go to: **http://localhost:5678**

First launch: Enter name, email, create password → Click 'Get started'

Common Errors & Fixes

Error	Fix
command not found: nvm	Run the export command in Step 2, or open a fresh terminal
ModuleNotFoundError: distutils	You are on Node v25. Switch to Node 20 using nvm (Steps 5-6)
command not found: n8n	Re-install: <code>npm install -g n8n</code> (make sure Node 20 is active)
Port 5678 already in use	Run: <code>n8n start --port 5679</code>

ACTIVITY: Installation Checkpoint:

After completing the steps above, verify your setup by filling in:

My Node.js version (`node -v`): _____

My npm version (`npm -v`): _____

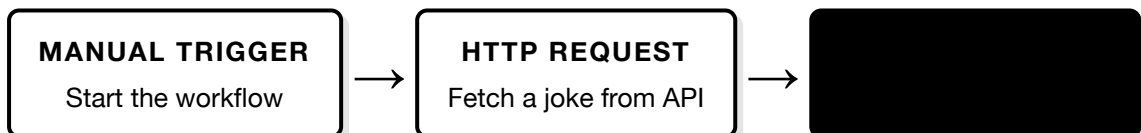
n8n is running at URL: _____

My n8n account email: _____

Part 5: Your First Workflow – The Joke Explainer

Goal: Build a 3-node workflow that fetches a random joke from an API and sends it to Groq AI to explain the joke in a fun way. **Zero code required!**

Workflow Overview



Step 1 – Create a new workflow

Go to `http://localhost:5678` → Click '**Start from scratch**' → You land on the blank canvas.

Step 2 – Add Manual Trigger node

Click the '+' button on the canvas → Search for: **Manual Trigger** → Click to add it.

💡 This is like the 'Run' button – it manually starts the workflow when clicked.

Step 3 – Add HTTP Request node

Click '+' after Manual Trigger → Search for: **HTTP Request** → Add it.

In the URL field, paste:

```
https://official-joke-api.appspot.com/random_joke
```

Click '**Execute step**' to test.

✓ OUTPUT will show: `type`, `setup`, `punchline`, `id` fields

OBSERVATION: Compare this to the Python equivalent — same result, but with code:

```
import requests
response = requests.get("https://official-joke-api.appspot.com/random_joke")
print(response.json())
# In n8n, the HTTP Request node does ALL of the above — zero code needed!
```

Step 4 — Get your FREE Groq API Key

1. Go to: <https://console.groq.com>
2. Sign up or log in
3. Click '**API Keys**' in the left sidebar
4. Click '**Create API Key**'
5. Copy the key — it looks like: `gsk_XXXXXXXXXXXX`

Step 5 — Add Basic LLM Chain node (with Groq)

Click '+' after HTTP Request → Search for: **Basic LLM Chain** → Add it.

n8n will also auto-add a '**Groq Chat Model**' sub-node below it.

If any extra nodes appeared (like 'When chat message received'), delete them.

Step 6 — Configure Basic LLM Chain

Click on the Basic LLM Chain node:

1. Change '**Source for Prompt**' from 'Connected Chat Trigger Node' to '**Define below**'
2. In the Prompt field, paste:

```
Here is a joke:
Setup: {{ $json.setup }}
Punchline: {{ $json.punchline }}
Explain why this joke is funny in a simple and fun way!
```

💡 `{{ $json.setup }}` and `{{ $json.punchline }}` are n8n variables that pull data from the previous node (HTTP Request).

Step 7 — Configure Groq Chat Model

Click on the '**Groq Chat Model**' node (orange circle below Basic LLM Chain):

1. Click '**Create new credential**'
2. Paste your Groq API key (`gsk_XXXXXXXXXXXX`)
3. Click **Save**
4. Set Model to: **llama3-8b-8192**

Step 8 — Run the full workflow!

Close all node panels → Click '**Execute workflow**' button at the bottom → Wait a few seconds.

Click on 'Basic LLM Chain' node to see Groq's response in the OUTPUT panel.

✅ All three nodes show green checkmarks = workflow ran successfully!

ACTIVITY: Workflow Verification:

What joke did you get?

Setup: _____

Punchline: _____

What did Groq say about it?

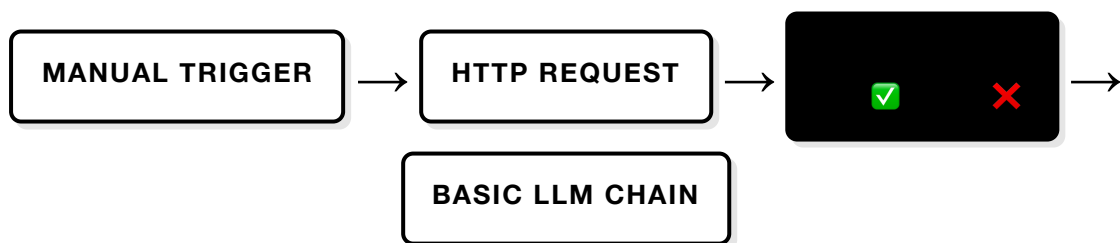
How many nodes are in your workflow? _____

KEY INSIGHT: You just built your first AI-powered workflow — **without writing a single line of code**. The same thing in Python would require importing `requests`, `groq`, handling API keys, error handling, and more. n8n did it all visually.

Part 6: IF Conditions — Smart Decision Making

Goal: Add an IF node to filter jokes. Only jokes of type `'general'` will pass through to Groq. All other types (programming, knock-knock) will be blocked.

Updated Workflow



About Joke Types

Type	Example	IF Result
general	Why do bees have sticky hair? Honey combs!	✅ true → goes to Groq
programming	Why do Java devs wear glasses? No C#!	❌ false → blocked
knock-knock	Knock knock... Who's there?	❌ false → blocked

Step 1 — Disconnect HTTP Request from Basic LLM Chain

Hover over the arrow connecting them → Click the ❌ delete icon on the line.

Step 2 — Add IF node after HTTP Request

Click '+' on the right side of HTTP Request → Search for: **IF** → Add it.

You will see two outputs: **'true'** and **'false'**.

Step 3 — Configure the IF condition

Field	Value
Value 1	{{ \$json.type }}
Operation	is equal to
Value 2	general

Step 4 — Connect IF (true) → Basic LLM Chain

Drag from the **'true'** output of the IF node to the Basic LLM Chain node. Leave **'false'** unconnected.

Step 5 — Test TRUE branch

URL: https://official-joke-api.appspot.com/random_joke

Click 'Execute workflow' → If type is 'general' → true branch lights up green → Groq explains.

Step 6 — Test FALSE branch

Change URL to: <https://official-joke-api.appspot.com/jokes/programming/random>

Click 'Execute workflow' → type is 'programming' → does NOT equal 'general' → false → Groq does NOT run.

OBSERVATION: The Python equivalent of the IF node:

```
joke = requests.get('https://...random_joke').json()

# IF node does exactly this:
if joke['type'] == 'general':
    send_to_groq(joke)
else:
    pass

# TRUE branch
# → Basic LLM Chain runs
# FALSE branch
# → workflow stops here
```

If the API returns `{"type": "knock-knock", "setup": "Knock knock"}`, what happens?

- A. The Basic LLM Chain node runs and Groq explains the joke
- B. The workflow takes the FALSE branch and stops — Groq never runs
- C. n8n crashes with an error
- D. The IF node is skipped entirely

Hint: The IF node checks: is type equal to "general"? "knock-knock" ≠ "general", so which branch fires?

Part 7: Schedule Trigger — Automate Like a Pro

Goal: Replace the Manual Trigger with a Schedule Trigger so the workflow runs **automatically every day at 8 AM** — no clicking needed!

THE ANALOGY: Alarm Clock vs. Manual Wake-Up

Manual Trigger = You decide to wake up and look at the time.

Schedule Trigger = Your alarm clock rings at 8 AM every day, no matter what.

New Workflow (build from scratch)



Step 1 — Create a new blank workflow

Go to `http://localhost:5678` → Click '+' to create a new workflow.

Step 2 — Add Schedule Trigger

Click 'Add first step...' → Search for: **Schedule** → Click 'Schedule Trigger'.

Step 3 — Configure the Schedule Trigger

Field	Value
Days Between Triggers	1 (runs every day)
Trigger at Hour	8 (8 AM)
Trigger at Minute	0 (at :00 exactly)

💡

Close the panel and continue adding the remaining nodes (HTTP Request → IF → Basic LLM Chain) with the same settings from Parts 5 and 6.

Important: Timezone Setup for IST

The Schedule Trigger uses the **server's timezone** — NOT your local timezone. By default it may use `America/New_York`.

Set IST timezone — Two Methods

Method 1: Quick Inline (for testing)

Run n8n with the timezone set in the same command — no file changes needed:

`GENERIC_TIMEZONE="Asia/Kolkata" n8n start`

💡

This sets the timezone *only for this session*. You must type it every time you start n8n.

Method 2: Permanent Fix (recommended)

Add it to your shell config so it's always set:

- 1. Stop n8n in your terminal with Ctrl + C
- 2. Add timezone permanently:

```
echo 'export GENERIC_TIMEZONE="Asia/Kolkata"' >> ~/.zshrc
source ~/.zshrc
```

- 3. Start n8n normally: n8n start

Now every future n8n start will automatically use IST — no extra typing needed.

Timezone	n8n Setting	When is 8 AM IST?
India (IST)	Asia/Kolkata	8:00 AM
UTC	UTC	2:30 AM
US Eastern	America/New_York	10:30 PM (prev day!)

Local vs Cloud Limitation

Local n8n (your Mac)	Cloud n8n (production)
Mac must be ON and awake	Runs 24/7 on a server
n8n terminal must be running	Never misses a trigger
If Mac sleeps → trigger is missed!	Options: Railway, Render, n8n Cloud
Good for: learning and testing only	Good for: real production workflows

OBSERVATION: The Python equivalent of Schedule Trigger:

```
import schedule, time

def run_workflow():
    joke = fetch_joke()           # HTTP Request node
    if joke['type'] == 'general': # IF node
        explain(joke)             # Basic LLM Chain

# Schedule Trigger does this:
schedule.every().day.at('08:00').do(run_workflow)
while True: schedule.run_pending(); time.sleep(60)
```

ACTIVITY: Fill in the Blanks:

The Schedule Trigger replaces the _____ Trigger.

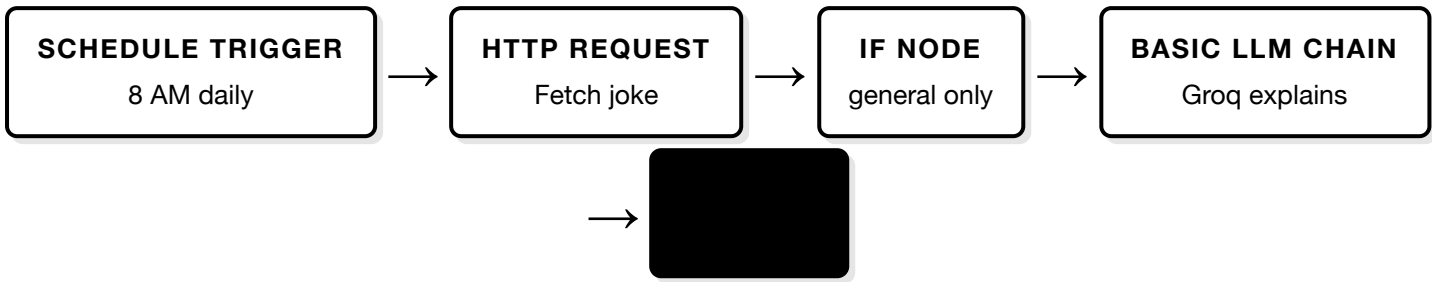
To set IST timezone, you set the environment variable _____ to _____.

A Schedule Trigger on your local Mac will **NOT** fire if _____.

Part 8: Gmail Integration — Send Emails Automatically

Goal: Add a Gmail node at the end so the Groq-explained joke is automatically emailed every morning. This requires setting up Google Cloud OAuth credentials.

Complete Final Workflow



Phase A: Google Cloud Console Setup (One Time)

Step 1 — Create a Google Cloud Project

- 1. Open: <https://console.cloud.google.com>
- 2. Click 'Select a project' → 'New Project'
- 3. Project name: `n8n-gmail` → Click 'Create'

Step 2 — Enable Gmail API

Go to **APIs & Services** → **Library** → Search for: **Gmail API** → Click 'Enable'

Step 3 — Configure OAuth Consent Screen

- 1. Click 'OAuth consent screen' → Click 'Get started'
- 2. App name: `n8n-gmail` , User support email: your Gmail
- 3. Audience: select **External** → Contact info: your Gmail → Click 'Create'

Step 4 — Create OAuth Client ID

- 1. Click 'Create OAuth client' → Type: **Web application** → Name: `n8n`
- 2. Under 'Authorised redirect URIs' → Add URI:

`http://localhost:5678/rest/oauth2-credential/callback`

Click 'Create' → **Copy BOTH Client ID and Client Secret immediately!**

Credential	Looks like
Client ID	<code>xxxxxx.apps.googleusercontent.com</code>
Client Secret	<code>GOCSPX-xxxxxxxxxxxxxxxxxx</code>

OBSERVATION: Missed the popup? Here's how to get your Client Secret later:

1. In Google Cloud Console, click '**Clients**' on the left sidebar (under APIs & Services)
2. You'll see your "**n8n**" client listed in the table
3. Click on it to open the details
4. Both **Client ID** and **Client Secret** are displayed on this page — copy them from here

Phase B: Connect Gmail in n8n

Step 5 — Add Gmail node to the workflow

Click '+' after Basic LLM Chain → Search for: **Gmail** → Click 'Send a message'.

Step 6 — Connect Gmail credentials

1. Click 'Select Credential' → 'Create new credential'
2. Select 'OAuth2 (recommended)'
3. Paste Client ID and Client Secret → Click Save
4. Click 'Connect' / 'Sign in with Google' → Allow permissions

✅ You will see: '*Account connected*'

Step 7 — Configure the Gmail node

Field	Value
To	your-email@gmail.com
Subject	Your Daily Joke!
Email Type	Text
Message	{{ \$json.text }}

💡 {{ \$json.text }} pulls Groq's joke explanation from the Basic LLM Chain node.

Step 8 — Test & Publish

1. Click '**Execute step**' → Check your Gmail inbox!
2. Click '**Publish**' at the top right → Workflow is now active and auto-runs daily!

✅ Email received with joke explanation = Part 8 complete!

⚠️ WARNING: Your Mac must be ON and n8n must be running in Terminal for the schedule to trigger automatically. For production use, deploy n8n to a cloud server.

OBSERVATION: The complete Python equivalent of the entire workflow:

```

import requests, schedule, time
from groq import Groq
import smtplib

def run_workflow():
    # HTTP Request node
    joke = requests.get('https://official-joke-api.appspot.com/random_joke').json()
    # IF node
    if joke['type'] == 'general':
        # Basic LLM Chain + Groq
        explanation = groq.explain(joke['setup'], joke['punchline'])
        # Gmail node
        send_email(to='you@gmail.com', body=explanation)

# Schedule Trigger
schedule.every().day.at('08:00').do(run_workflow)
while True: schedule.run_pending(); time.sleep(60)

```

n8n did all of the above with ZERO lines of code.

Part 9: Terminology Matching — Speak n8n Fluently

Match each n8n term (left) with its correct definition (right). Write the letter next to the number.

#	Term		Letter	Definition
1	Node	→	_____	A. A connected sequence of nodes that perform a task end-to-end
2	Workflow	→	_____	B. Securely stored API keys and passwords for external services
3	Trigger	→	_____	C. A single unit of work that performs one specific action
4	Credentials	→	_____	D. The syntax to access data from a previous node: {{ \$json.x }}
5	Expression	→	_____	E. The special node that starts a workflow (manual, schedule, webhook)
6	INPUT panel	→	_____	F. Shows the data a node produces after execution
7	OUTPUT panel	→	_____	G. Shows the data flowing INTO a node from the previous node
8	OAuth2	→	_____	H. An authentication protocol used by Google/Gmail to grant access securely

Which expression correctly pulls the "setup" field from an HTTP Request node's output?

- A. \$json.setup
- B. {{ \$json["setup"] }}
- C. {{ \$json.setup }}
- D. json.setup()

Hint: n8n expressions always use double curly braces {{ }} and start with \$json followed by a dot and the field name.

Part 10: All 5 Parts Summary — What You Built

Part	Topic	What You Learned
✔ Part 4	Installation	nvm, Node 20 LTS, <code>npm install n8n</code> , <code>localhost:5678</code>
✔ Part 5	First Workflow	Manual Trigger → HTTP Request → Basic LLM Chain + Groq
✔ Part 6	IF Condition	Filter jokes by type — true/false branching
✔ Part 7	Schedule Trigger	Auto-run at 8 AM IST daily — cron job concept
✔ Part 8	Gmail Integration	Google OAuth setup, Gmail node, send email automatically

KEY INSIGHT: You have built a complete no-code AI agent that runs automatically every morning, fetches a joke, filters it with logic, gets Groq to explain it, and emails it to you — **all without writing a single line of code!**

ACTIVITY: Reflection:

What was the most surprising thing you learned about no-code tools?

If you could build any automation with n8n, what would it be? Describe it in 2-3 sentences:

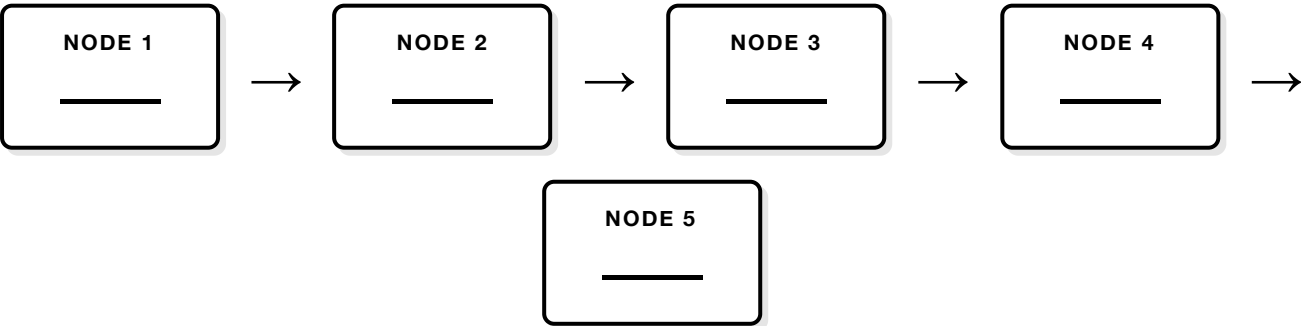
Final Challenge: Design Your Own n8n Workflow

THE CHALLENGE: "The Smart Morning Briefing"

Your manager wants a **daily morning briefing email** sent at 9 AM IST. The email should contain a weather update from a public weather API, summarized by an AI (Groq), and only sent on **weekdays** (Monday–Friday). On weekends, nothing should happen.

Task 1: Design the Workflow

Draw the nodes and connections for this workflow. Use the boxes below:



Task 2: Configure the IF Node

What condition would you set in the IF node to check if today is a weekday?

Field	Your Answer
Value 1	_____
Operation	_____
Value 2	_____

Hint: The Schedule Trigger outputs a "day" field. Saturday = 6, Sunday = 0. Think about how to check "not Saturday AND not Sunday".

Task 3: Write the LLM Prompt

What prompt would you give to the Basic LLM Chain to summarize the weather data?

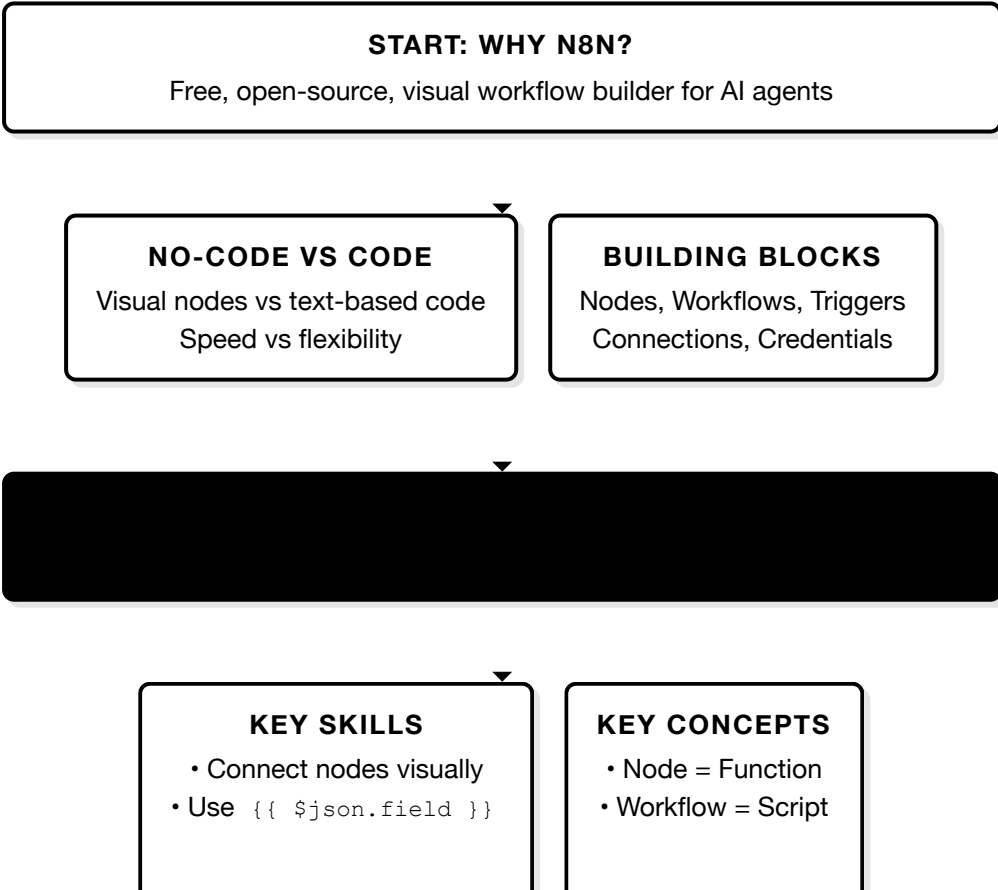
YOUR PROMPT

Task 4: Python Equivalent

Write the Python equivalent of your entire workflow in pseudocode (don't worry about exact syntax):

PYTHON PSEUDOCODE

Recap: Your n8n Learning Journey



- Set up API credentials
- Configure triggers

- IF = Conditional
- Schedule = Cron

RESULT

A fully automated AI agent that fetches, filters, explains, and emails — zero code!

Quick Reference Links

Resource	URL
n8n Documentation	https://docs.n8n.io
Groq Console	https://console.groq.com
Google Cloud Console	https://console.cloud.google.com
n8n Community	https://community.n8n.io
Joke API (for testing)	https://official-joke-api.appspot.com/random_joke